Browser-in-the-Loop: Reinforcement Fine-Tuning LLM Agents for Web Form Filling

Muhammad Enrizky Brillian
Department of Computer and Mathematical Sciences
University of Toronto Scarborough
University of Toronto
billy.suharno@mail.utoronto.ca

Abstract

Training a language model to automate web browsing is difficult because small mistakes cascade: if the model clicks the wrong button, the entire page changes and all subsequent actions fail. Supervised fine-tuning (SFT) teaches a model to copy human demonstrations, but it never learns how to recover from its own mistakes. We investigate whether reinforcement learning—specifically, Group Relative Policy Optimization (GRPO)—can improve a fine-tuned model by letting it practice on real web forms and learn from the outcomes. We use QLoRA (a memory-efficient fine-tuning method) to adapt Qwen3-8B (an 8-billion-parameter language model) for form-filling tasks on the FormFactory benchmark, which contains 1,250 tasks across eight domains such as healthcare, finance, and legal compliance. Our two-phase pipeline first trains the model via SFT on 992 demonstrations to learn the correct output format, then applies online GRPO where the model generates action plans, executes them in a real browser, and receives a reward score based on how well it filled and submitted the form. On 124 held-out validation tasks, GRPO achieves an average reward of 0.670 compared to SFT's 0.614—a 9.1% improvement. On a separate 124-example test split, GRPO improves by 5.4% (0.669 vs. 0.635). We find that SFT initialization is necessary: without it, GRPO cannot discover the structured action format and produces zero reward. We also show that naïve SFT with a high learning rate memorizes training patterns and degrades generalization; a careful combination of reduced learning rate, early stopping, and data shuffling yields the best SFT baseline for GRPO to build upon.

1 Introduction

Web automation is a high-impact application of large language model (LLM) agents. Tasks such as filling out forms, navigating multi-page workflows, and extracting information from dynamic web pages require models to generate sequences of browser actions based on the page's current structure—technically called the Document Object Model (DOM), which is the browser's internal representation of all elements on a web page (buttons, text fields, dropdowns, etc.). Despite rapid progress in LLM capabilities, even state-of-the-art models frequently fail on multi-step web workflows due to *compounding error*: a single incorrect action changes the page in unexpected ways, and every subsequent action is now wrong because the model has never seen this new page state during training. For example, if a model accidentally clicks "Next Page" instead of filling a required field, the form advances to a new page where none of the remaining planned actions make sense (Yao et al., 2022).

Supervised Fine-Tuning (SFT) is the standard approach for adapting LLMs to domain-specific tasks. In SFT, the model learns by copying expert demonstrations: given a web page and an instruction like "fill out this medical form," the model is trained to predict the exact sequence of actions a human

Preprint.

would take. While SFT produces models that can faithfully reproduce training trajectories, it has a critical weakness: it never teaches the model what to do when something goes wrong. If the model makes a mistake and the page enters a state it has never seen before, SFT offers no guidance for how to recover.

Reinforcement Fine-Tuning (RFT) addresses this limitation by letting the model learn from its own experience through trial and error. Instead of copying demonstrations, the model tries different action sequences on real web forms, observes whether each attempt succeeds or fails, and gradually adjusts its behavior to do more of what works and less of what does not. This trial-and-error process enables the model to discover strategies—such as alternative ways to submit a form—that are absent from the fixed demonstrations used in SFT.

However, the most popular reinforcement learning algorithm for language models—Proximal Policy Optimization (PPO) (Schulman et al., 2017)—requires training two neural networks at once: the main model (called the *policy*, which generates actions) and a second model (called the *critic* or *value function*, which estimates how good the current state is). Running two large models simultaneously substantially increases the GPU memory needed, making PPO impractical on a single consumer-grade GPU.

Group Relative Policy Optimization (GRPO) (Shao et al., 2024) solves this problem by eliminating the critic entirely. Instead of relying on a separate network to judge performance, GRPO uses a simple comparison-based approach: for each input, it generates G different candidate outputs (e.g., $G = 4$ different action plans), scores all of them, and then determines which ones are relatively better or worse *within the group*. A candidate that scores above the group average receives a positive "advantage" (meaning: do more of this), while one that scores below average receives a negative advantage (meaning: do less of this). This removes the need for the critic network, significantly reducing memory requirements and making reinforcement fine-tuning feasible on a single GPU.

In this work, we apply GRPO to fine-tune Qwen3-8B (Yang et al., 2025) for form-filling tasks on the FormFactory benchmark (Li et al., 2025). Our pipeline has two phases. In **Phase 1 (SFT)**, we fine-tune the model on 992 expert demonstrations using QLoRA (Dettmers et al., 2023)—a memory-efficient method that freezes most of the model's parameters and trains only small adapter layers (explained in Section 3.2)—so the model learns to output structured action plans (e.g., "Step 1: Type 'Name' with 'John"'). In **Phase 2 (GRPO)**, we let the model practice on live web forms: it generates action plans, a real headless browser executes them, and the model receives a reward score based on how well the form was filled and whether it was successfully submitted. Over many practice rounds, the model learns to produce better action plans.

Contributions. We make the following contributions:

1. We show that SFT initialization is **necessary** for GRPO: without first teaching the model the action format via SFT, GRPO produces zero reward because the model outputs free-form reasoning text instead of executable action plans.

2. We provide a head-to-head comparison of SFT-only vs. SFT+GRPO on 248 held-out tasks. GRPO achieves 9.1% higher average reward on the validation split (0.670 vs. 0.614) and 5.4% higher on the test split (0.669 vs. 0.635).

3. We design a composite reward function that gives the model credit for partial progress—not just binary success/failure—by combining three scores: whether the form was submitted (40% weight), how accurately each field was filled (40% weight), and what fraction of planned actions actually executed in the browser (20% weight).

4. We release the complete training pipeline—SFT trainer, GRPO trainer with live browser execution, reward functions, and evaluation scripts—as well as the trained model weights to enable reproducible research.[1]

[1]Code: `https://github.com/billy-enrizky/openbrowser-ai`. Models: `https://huggingface.co/billyenrizky/Qwen3-8B-FormFactory-SFT-LoRA` and `https://huggingface.co/billyenrizky/Qwen3-8B-FormFactory-GRPO-LoRA`.

2 Related Work

2.1 Web Navigation Agents

Several benchmarks and methods have been proposed for training LLMs to interact with web pages. WebShop (Yao et al., 2022) provides a simulated e-commerce environment where an agent must navigate product pages and complete purchases. Mind2Web (Deng et al., 2023) contributes a large-scale dataset of real-world web tasks for training generalist web agents. WebArena (Zhou et al., 2024) offers a realistic, self-hosted benchmark for evaluating agents on complex multi-step web workflows. More recently, several works have applied reinforcement learning directly to web agents: AgentQ (Putta et al., 2024) combines tree search with preference optimization for web navigation, DigiRL (Bai et al., 2024) applies online RL to device-control agents, and WebRL (Qi et al., 2024) trains web agents using a self-evolving curriculum. These works primarily evaluate models in zero-shot or few-shot settings, or on benchmarks different from FormFactory, without fine-tuning on form-filling specifically.

2.2 Form-Filling Benchmarks

FormFactory (Li et al., 2025) is the benchmark we use in this paper. It provides a collection of realistic web forms (medical intake forms, banking applications, legal filings, etc.) together with an evaluation system that checks whether each form was correctly filled and submitted. Importantly, their evaluation of state-of-the-art commercial LLMs (GPT-4o, Gemini, etc.) found that *no model surpasses 5% accuracy* when tested zero-shot without an agentic browser framework (i.e., using simple prompting without structured action execution or task-specific training). When paired with an agentic browser framework, these same commercial models achieve 98–100% success rates (Table 1), but at the cost of expensive API calls for every inference. This contrast motivates our approach: instead of relying on costly commercial APIs, we fine-tune a smaller model specifically for this task and then improve it further with reinforcement learning.

2.3 Reinforcement Learning for LLMs

Reinforcement Learning from Human Feedback (RLHF) (Ouyang et al., 2022) pioneered the use of RL to align language models with human preferences, and PPO (Schulman et al., 2017) has been the dominant algorithm for this purpose. However, as discussed above, PPO requires a separate critic network that substantially increases memory usage. DeepSeekMath (Shao et al., 2024) introduced GRPO, which removes the critic by instead comparing multiple candidate outputs against each other—a much simpler and more memory-efficient approach built on the classic REINFORCE (Williams, 1992) algorithm. GRPO matches or exceeds PPO's performance on mathematical reasoning benchmarks while significantly reducing GPU memory requirements. DeepSeek-R1 (DeepSeek-AI, 2025) further showed that RL can induce emergent reasoning capabilities in LLMs. Our work is the first to apply GRPO to interactive web environments, where the reward signal comes from actually executing actions in a real browser rather than from text comparison alone.

2.4 Parameter-Efficient Fine-Tuning

Fine-tuning an 8-billion-parameter model normally requires enormous GPU memory because every parameter must be stored in 16-bit (bfloat16) format, requiring roughly 16 GB for the weights alone—plus additional memory for gradients and optimizer states. QLoRA (Dettmers et al., 2023) solves this problem with two ideas: (1) it *quantizes* (compresses) the base model's weights from 16-bit down to 4-bit, shrinking memory usage by roughly $4\times$; and (2) instead of updating all billions of parameters, it freezes the compressed base model and trains only small "adapter" matrices that are added on top. These adapters use Low-Rank Adaptation (LoRA) (Hu et al., 2021), which reduces the number of trainable parameters from billions to just a few million. Together, these techniques allow us to fine-tune Qwen3-8B on a single NVIDIA A10G GPU with only 24 GB of memory.

3 Background

3.1 Group Relative Policy Optimization (GRPO)

GRPO (Shao et al., 2024) is a reinforcement learning algorithm for language models that eliminates the need for a critic network. The core idea is simple: instead of training a separate model to judge how good each output is, GRPO generates multiple outputs for the same input and compares them against each other.

Concretely, for each input prompt x, GRPO samples a group of G candidate outputs $\{y_1, y_2, \ldots, y_G\}$ from the current model (called the *policy* π_θ). A reward function $R(x, y_i)$ scores each output. To determine which outputs are "good" or "bad" relative to the group, GRPO computes an *advantage* for each candidate by subtracting the group's mean reward and dividing by the group's standard deviation:

$$A_i = \frac{R(x, y_i) - \mu_G}{\sigma_G} \tag{1}$$

where $\mu_G = \frac{1}{G}\sum_{j=1}^{G} R(x, y_j)$ is the average reward in the group and $\sigma_G = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(R(x, y_j) - \mu_G)^2}$ is the standard deviation. An output that scores above the group average gets a positive advantage (the model should produce more outputs like this), while one that scores below average gets a negative advantage (the model should avoid outputs like this).

The model is then updated to increase the probability of high-advantage outputs and decrease the probability of low-advantage ones, using a clipped objective with a penalty that prevents the model from changing too much from a reference policy π_{ref} (typically the model before GRPO training):

$$\mathcal{L}_{\text{GRPO}} = \mathbb{E}_{x,\{y_i\}}\left[\frac{1}{G}\sum_{i=1}^{G} \min\left(\rho_i A_i,\ \text{clip}(\rho_i, 1{-}\epsilon, 1{+}\epsilon) A_i\right) - \beta \cdot D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})\right] \tag{2}$$

where $\rho_i = \frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)}$ is the *importance ratio* (how much more or less likely the current model is to produce output y_i compared to the old policy that generated the candidates), ϵ is the clip range that limits how much the policy can change in a single update, and β is the KL penalty coefficient that controls how strongly the model is discouraged from deviating too far from the reference policy π_{ref}. The $\min(\cdot)$ and $\text{clip}(\cdot)$ prevent excessively large updates that could destabilize training.

We measure how much the current model has changed from the reference using KL divergence (Kullback–Leibler divergence), a standard measure of how different two probability distributions are. A KL divergence of zero means the two models behave identically; larger values mean they have diverged more. We use the Schulman k3 estimator, which guarantees non-negative values (important for numerical stability):

$$D_{\text{KL}}^{\text{k3}} = r - \log r - 1 \geq 0, \quad \text{where } r = \frac{\pi_{\text{ref}}(y|x)}{\pi_\theta(y|x)} \tag{3}$$

3.2 QLoRA

QLoRA (Dettmers et al., 2023) combines two ideas to make fine-tuning large models memory-efficient:

4-bit quantization: The base model's weight matrices $W_0 \in \mathbb{R}^{d\times k}$ (where d and k are the layer dimensions) are compressed from 16-bit (bfloat16) to 4-bit NormalFloat (NF4) format and frozen—they are never modified during training.

Low-rank adapters: Instead of updating the full weight matrix (which would require storing gradients for billions of parameters), QLoRA adds two small matrices $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$, where r is much smaller than both d and k (in our case, $r = 16$). The effective weight becomes:

$$W = W_0^{\text{NF4}} + \frac{\alpha}{r} \cdot BA \tag{4}$$

where α is a scaling factor. During training, only B and A are updated. Because r is small, these adapters contain only a few million parameters—compared to the billions in the frozen base model—making fine-tuning feasible on a single GPU.

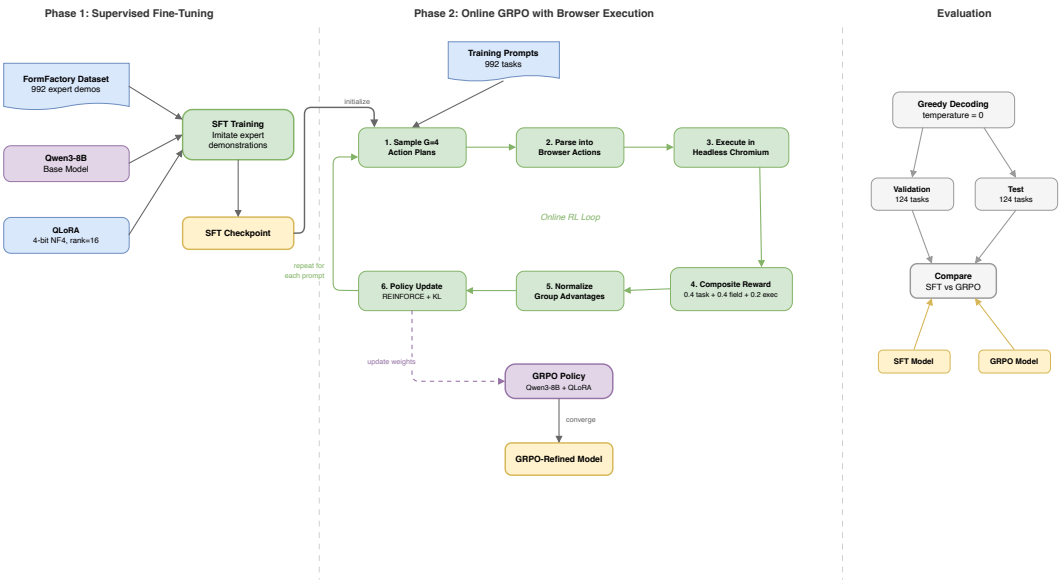


Figure 1: Overview of our two-phase training pipeline. Phase 1: Supervised Fine-Tuning (SFT) on 992 FormFactory demonstrations teaches the model to produce structured action plans. Phase 2: Online GRPO with browser execution refines action quality through trial-and-error interaction with live forms. The composite reward signal combines task completion, field accuracy, and execution completeness.

3.3 Qwen3-8B

Qwen3 (Yang et al., 2025) is a family of language models developed by Alibaba. We use Qwen3-8B, an 8-billion-parameter model where all parameters are active on every input (called a "dense" model, as opposed to sparse mixture-of-expert architectures where only a subset of parameters are used per input). A relevant feature of Qwen3 is its "thinking mode": by default, the model first reasons step-by-step inside `<think>...</think>` tags before producing its final answer—similar to how a student might show their work before writing a solution. While this is useful for math and reasoning tasks, it is counterproductive for our form-filling task because we need the model to directly output action plans (e.g., "Step 1: Type 'Name' with 'John' ") without wasting tokens on intermediate reasoning. We suppress thinking mode by prepending empty think tags (`<think>\n</think>\n`) to the model's output, which signals the model to skip the reasoning phase and produce actions immediately.

4 Methodology

4.1 Problem Formulation

We frame the form-filling task as follows: given a natural language instruction $\mathcal{I}$ that describes which form to fill and what values to enter (e.g., "Fill out the patient intake form with name John Doe, date of birth 01/15/1990, insurance ID ABC123"), the model must produce a sequence of browser actions $a_1, a_2, \ldots, a_T$ that correctly fills every field and submits the form. Each action is one of three types:

- **Type**(field_name, value): Enter text into a form field.
- **Select**(field_name, option): Choose an option from a dropdown.
- **Click**(element): Click a button or element.

The agent generates the entire action plan as a structured text sequence, producing one word (token) at a time from left to right, where each word depends on all previously generated words:

```
Step 1: Type "First Name" with "John"
```

```
Step 2: Type "Last Name" with "Doe"
Step 3: Select "Country" with "Canada"
Step 4: Click "Submit"
```

A key challenge is that the field names in the model's output (e.g., "First Name") may not exactly match the field labels in the web page's HTML code (e.g., `first_name` or `fname`). Our action parser bridges this gap using a multi-level matching strategy that tries progressively fuzzier matches: first exact match, then ignoring underscores and spaces, then stripping all special characters, then checking common abbreviations (e.g., "Date of Birth" $\rightarrow$ "dob"), and finally substring containment. Once each field name is matched to a page element, the parser executes each action in a headless (invisible) Chromium browser using the Playwright automation library.

4.2 Phase 1: Supervised Fine-Tuning (SFT)

The goal of Phase 1 is to teach the model the correct output format: given a task instruction, produce a step-by-step action plan. Each training example pairs an instruction (which includes the form URL, a description of the task, and a list of fields and their target values) with a ground-truth action sequence written by a human demonstrator.

We apply QLoRA to Qwen3-8B with the following configuration:

- **Quantization:** 4-bit NormalFloat (NF4) with double quantization (an additional compression step that further reduces memory) and bfloat16 (a 16-bit floating-point format) for intermediate computations.
- **LoRA:** Rank $r = 16$, scaling factor $\alpha = 32$, dropout $p = 0.05$, applied to all linear layers.
- **Optimizer:** Paged AdamW 8-bit (a memory-efficient variant of the Adam optimizer) with weight decay 0.01 and a warmup ratio of 0.05 (the learning rate starts near zero and gradually increases over the first 5% of training steps, preventing unstable early updates).
- **Training:** Batch size 2 with gradient accumulation over 8 steps (the model processes 2 examples at a time but accumulates gradients over 8 batches before updating weights, giving an effective batch size of $2 \times 8 = 16$). Maximum sequence length is 512 tokens. We tune the learning rate and number of training steps as ablation variables (Section 6.3); the best configuration uses lr=10^{-4} for 30 steps ($\sim$0.48 epochs) with shuffled data.
- **Label masking:** During training, we only compute the loss on the action sequence tokens (the model's response), not on the instruction tokens. This means the model is trained to *generate* action plans, not to memorize instructions.

4.3 Phase 2: Online GRPO with Browser Execution

Starting from the SFT checkpoint, we run online GRPO where the model interacts with live FormFactory forms via a headless browser (Algorithm 1). For each training prompt:

1. Sample $G = 4$ candidate action plans from the model using randomized decoding: *temperature* $T = 1.0$ controls overall randomness (higher = more random), and *top-p*$= 0.95$ means the model samples from the smallest set of tokens whose combined probability is at least 95%, ignoring very unlikely tokens.
2. Parse each plan into a sequence of browser actions using the multi-level matching action parser.
3. Execute each action sequence in a headless Chromium browser against the FormFactory form.
4. Compute a composite reward for each candidate based on the browser execution outcome.
5. Normalize advantages within the group and update the policy via REINFORCE with KL penalty.

Simplified implementation. The full GRPO objective (Equation 2) includes a clipping mechanism inherited from PPO, which requires storing the probability of every token under the old policy. To save memory—which is already limited because we run a real browser alongside the model—we use a

Algorithm 1 Online GRPO with Browser Execution

Require: SFT-initialized policy π_θ, reference policy π_{ref}, training prompts $\mathcal{D}$, group size G

for each prompt $x \in \mathcal{D}$ **do**
  Sample G action plans: $\{y_1, \ldots, y_G\} \sim \pi_\theta(\cdot|x)$
  for $i = 1$ to G **do**
    Parse y_i into browser actions $\{a_1^{(i)}, \ldots, a_T^{(i)}\}$
    Execute actions in headless Chromium
    Compute reward $r_i = R(x, y_i, \text{browser_outcome})$
  end for
  if $\text{Var}(\{r_1, \ldots, r_G\}) < 0.01$ **then**
    Skip gradient update (low reward variance)
  else
    Compute advantages: $A_i = (r_i - \mu)/\sigma$ for $i = 1, \ldots, G$
    Update θ via REINFORCE with group advantages + KL penalty
  end if
end for

simpler variant based on the classic REINFORCE algorithm (Williams, 1992), which directly adjusts the model's parameters to make high-reward outputs more likely, combined with GRPO's group-relative advantage normalization. In plain terms, for each candidate output y_i, the training update is: *increase the probability of y_i if it scored above the group average, and decrease it if it scored below average, by an amount proportional to how far above or below it was.* We add a KL penalty (Equation 3) to prevent the model from changing too drastically from its initial SFT-trained behavior. We retain the GRPO terminology throughout because the key innovation of GRPO—comparing candidates within a group to determine advantages—is fully preserved; only the PPO-style clipping is omitted.

Skipping low-variance groups. When all G candidates in a group receive nearly identical rewards (variance < 0.01), the advantages are near-zero and the only active term in the loss is the KL penalty, which would push the model *backward* toward the reference policy without any useful learning signal. We therefore skip the gradient update entirely for such groups.

GRPO hyperparameters: Group size $G = 4$, learning rate 5×10^{-5}, KL coefficient $\beta = 0.1$, maximum output length 512 tokens, action timeout 10 seconds, minimum reward variance 0.01. Training stops automatically (early stopping) if the running average reward (computed over a window of 20 gradient updates) does not improve for 50 consecutive gradient updates.

4.4 Reward Design

A key design decision is how to score each attempt. The simplest approach—giving a reward of 1 if the form was submitted successfully and 0 otherwise—is too coarse: the model gets no credit for filling 9 out of 10 fields correctly, making it hard to learn from partial successes. Instead, we design a *composite* reward function with three components that together provide a more informative score:

$$R(x, y) = w_1 \cdot R_{\text{task}} + w_2 \cdot R_{\text{field}} + w_3 \cdot R_{\text{exec}} \tag{5}$$

where $w_1 = 0.4$, $w_2 = 0.4$, $w_3 = 0.2$ and:

Task Completion (R_{task}). A binary score: 1 if the form was successfully submitted, 0 otherwise. We detect success by checking whether the browser navigated to a confirmation page after the final action (e.g., the URL contains "success" or the page displays a confirmation message).

Field Accuracy (R_{field}). A continuous score (between 0 and 1) measuring how accurately each field was filled. For each field in the form, we compare the value the model entered against the correct value using string similarity (Python's `SequenceMatcher`, which measures the overlap between two strings). The overall field accuracy is the average across all fields:

$$R_{\text{field}} = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \text{SequenceMatcher}(v_f^{\text{pred}}, v_f^{\text{true}}) \tag{6}$$

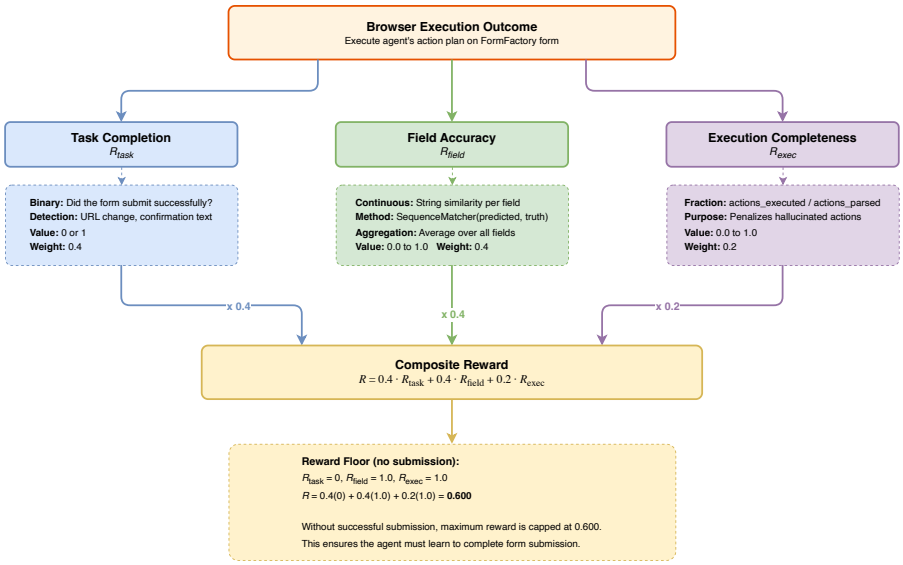


Figure 2: Composite reward function for form-filling tasks. Three components capture complementary aspects: task completion (binary submission success), field accuracy (continuous string similarity), and execution completeness (fraction of actions executed). When all fields are filled correctly but submission fails, the reward reaches a floor of $R = 0.600$.

where $\mathcal{F}$ is the set of form fields, v_f^{pred} is what the model entered, and v_f^{true} is the correct value. For example, if a form has 10 fields and the model fills 7 of them perfectly, this component would be approximately 0.7. This provides useful feedback even when the form is only partially completed.

Execution Completeness (R_{exec}). The fraction of the model's planned actions that the browser was able to actually carry out:

$$R_{\text{exec}} = \frac{\text{number of actions successfully executed}}{\text{total number of actions in the plan}} \tag{7}$$

For example, if the model plans 8 actions but 2 of them reference fields that do not exist on the page (causing the browser to skip them), the execution completeness would be $6/8 = 0.75$. This component penalizes the model for "hallucinating" actions that cannot be carried out.

4.5 Browser Environment

We use the Playwright automation library to control a headless (invisible) Chromium browser. The browser environment handles four tasks:

- **Field name matching:** The model might output "Type 'First Name' with 'John'," but the actual HTML form might label this field as `first_name`, `fname`, or `FirstName`. Our five-level matching strategy progressively relaxes the matching criteria: (1) exact lowercase match, (2) treating underscores, hyphens, and spaces as equivalent (so "First Name" matches "first-name"), (3) stripping all special characters (so "First Name" matches "firstname"), (4) looking up common abbreviations (e.g., "Social Security Number" $\rightarrow$ "ssn"), and (5) checking if one string contains the other.
- **Action execution:** Each action (typing text, selecting a dropdown option, or clicking a button) is executed in the browser with a 10-second timeout per action.
- **Success detection:** After all actions are executed, the environment checks whether the form was successfully submitted by looking for a confirmation page (e.g., a URL containing "success" or a "Thank you" message).
- **Form serving:** A local Flask web server (localhost:5050) hosts all FormFactory forms, generating HTML pages with various field layouts for evaluation.

5 Experimental Setup

5.1 Dataset

We use the FormFactory benchmark (Li et al., 2025), which contains 1,250 form-filling tasks spanning eight real-world domains: academic research, arts and creative, construction and manufacturing, finance and banking, healthcare and medical, legal compliance, professional business, and tech software. The benchmark includes 25 distinct form types (e.g., patient intake forms, loan applications, building permits). Each task specifies: (1) the URL of the form to fill, (2) the correct values for each field, and (3) a step-by-step action sequence showing how to fill and submit the form. A typical form has about 10 fields, including text inputs, dropdown menus, checkboxes, and submit buttons.

We exclude 10 tasks whose forms fail to render correctly in our local server, leaving 1,240 usable tasks. We split these into three non-overlapping sets: training (992 examples, 80%), validation (124 examples, 10%), and test (124 examples, 10%). The split is *stratified* by form type, meaning each of the 25 form types appears in all three sets proportionally (using random seed 42). Both SFT and GRPO train on the 992-example training set. We shuffle the training data to break apart sequential blocks of the same form type, which we found is critical for preventing memorization (Section 6.3).

5.2 Baselines

We compare our SFT+GRPO model against four baselines to answer different questions:

1. **Zero-shot Qwen3-8B:** The base Qwen3-8B model with no fine-tuning at all, simply prompted to produce form-filling actions. This tests whether the model can do the task "out of the box."
2. **Qwen3-8B SFT-only:** The model after Phase 1 (SFT on 992 demonstrations) but without Phase 2 (GRPO). This is our primary comparison—does GRPO actually help beyond SFT?
3. **Qwen3-8B GRPO without SFT:** GRPO applied directly to the base model, skipping SFT entirely. This tests whether SFT is truly necessary or whether GRPO can learn from scratch.
4. **Gemini-2.5-Flash and GPT-4o (API):** Two commercial models from Google and OpenAI, accessed via API using an agentic browser framework (Agent and CodeAgent modes) without any fine-tuning, to provide an upper-bound reference from much larger models.

5.3 Evaluation Metrics

We evaluate models using two metrics:

- **Nonzero Reward Rate:** The percentage of evaluation tasks where the model earns any reward at all ($R > 0$). A 100% nonzero rate means the model produces at least partially useful actions on every single task. A low nonzero rate means the model frequently generates completely unusable outputs.
- **Average Reward:** The mean composite reward (Equation 5) across all evaluation tasks, ranging from 0 (total failure) to 1 (perfect form filling and successful submission). This captures the overall quality of form filling, including partial credit for partially correct attempts.

All fine-tuned models are evaluated with **greedy decoding** (the model always picks the highest-probability next token, with no randomness). This ensures reproducible results: running the same model on the same form always produces the same output. We found this important because initial evaluations using random sampling (temperature=0.7) produced inconsistent results that made model comparison unreliable.

5.4 Compute Infrastructure

All training and evaluation runs on a single NVIDIA A10G GPU (24 GB VRAM) via AWS g5.xlarge instances on Anyscale Ray. QLoRA enables both SFT and GRPO to fit within the 24 GB memory budget. SFT training requires approximately 2–4 hours to make 2 full passes through the 992 training samples (each pass is called an *epoch*). Online GRPO requires approximately 4–8 hours per epoch

because each training step involves launching a real browser and executing actions, which is much slower than the text-only computations in SFT. Custom container images package all dependencies: PyTorch 2.x, Transformers 4.45+, PEFT 0.12+, BitsAndBytes 0.43+, and Playwright with Chromium.

6 Results

6.1 Main Results

Table 1 presents the main comparison across all model configurations. We report results on two held-out evaluation sets: a 124-example validation split and a 124-example test split.

Model	Phase	# Tasks	Decoding	Nonzero Rate	Avg Reward
Commercial API baselines					
Gemini-2.5-Flash (Agent)	–	100	–	98.0%	–
Gemini-2.5-Flash (CodeAgent)	–	100	–	100.0%	–
GPT-4o (Agent)	–	100	–	99.0%	–
GPT-4o (CodeAgent)	–	100	–	100.0%	–
Ablation baselines					
Qwen3-8B zero-shot	–	2	greedy	0.0%	0.000
Qwen3-8B GRPO (no SFT)	GRPO	25	greedy	0.0%	0.000
Fine-tuned models – Validation split (held-out)					
Qwen3-8B SFT-only	SFT	124	greedy	100.0%	0.614
Qwen3-8B SFT+GRPO	**GRPO**	**124**	**greedy**	**100.0%**	**0.670**
Fine-tuned models – Test split (held-out)					
Qwen3-8B SFT-only	SFT	124	greedy	100.0%	0.635
Qwen3-8B SFT+GRPO	**GRPO**	**124**	**greedy**	**100.0%**	**0.669**

Table 1: FormFactory evaluation results. "# Tasks" indicates the number of evaluation tasks used. Commercial API baselines use an agentic browser framework (Agent and CodeAgent modes) and report binary success rate on 100 tasks (a stricter metric than our composite reward, since binary success requires full form submission); without the agentic framework, these same models score below 5% (Li et al., 2025). Among fine-tuned Qwen3-8B models, SFT+GRPO achieves the highest average reward on both validation (0.670, +9.1% over SFT) and test (0.669, +5.4% over SFT). Without SFT initialization, GRPO produces 0% reward. All fine-tuned models evaluated with greedy decoding.

6.2 SFT Is Necessary for GRPO

A striking finding is that GRPO **completely fails** when applied to the base Qwen3-8B model without SFT. The reason is straightforward: the base model does not know it should output structured action plans like "Step 1: Type 'Name' with 'John'." Instead, it generates long reasoning paragraphs inside `<think>` tags—its default behavior for any prompt. Since the action parser cannot extract any valid actions from these reasoning traces, every attempt earns zero reward. We tested 25 training prompts with $G = 4$ candidates each (100 total attempts) and *every single one* produced zero reward. We also tested the zero-shot baseline on 2 prompts with the same result. We limited these ablations to small sample sizes because the failure is deterministic: the model always produces the same type of unusable output.

This reveals a fundamental insight: GRPO (and reinforcement learning in general) can only improve a model's existing capabilities—it cannot teach the model an entirely new output format from scratch. SFT serves as a *format teacher*: it transforms the model from a general-purpose text generator into one that produces structured action plans. Once the model knows the correct format, GRPO can then refine the *quality* of the actions within that format.

6.3 SFT Training Dynamics: Memorization vs. Generalization

We discovered that SFT is surprisingly sensitive to training settings and that more training is not always better.

The memorization problem. The FormFactory training data is organized in sequential blocks: 40 examples of one form type, then 40 of the next, and so on for all 25 form types. When we train on all 992 examples with a high learning rate (2×10^{-4}) for a full pass through the data (62 gradient steps), the training loss drops to near-zero (4.8×10^{-5}), meaning the model has essentially memorized every training example. Counter-intuitively, this *hurts* performance on new forms: the overfit model achieves only 94.4% nonzero rate and 0.426 average reward on validation—*worse* than a model trained on just 200 examples (98.4% nonzero, 0.486 reward).

Why memorization hurts. The problem is that a memorized model reproduces field names from the training data verbatim (e.g., it always outputs "Paper Title"). But validation forms may label the same field differently (e.g., `papertitle` or `paper_title`). While our action parser can handle some variation, a model that memorizes exact training patterns is more likely to produce field names that the parser cannot match. A model that has learned the *general concept* of what to type—rather than memorizing specific strings—generalizes better.

The fix. We make three changes: (1) reduce the learning rate to 10^{-4} so the model learns more gradually, (2) stop training early at 30 steps (less than half an epoch, meaning the model sees less than half the data), and (3) shuffle the training examples randomly so the model does not see 40 consecutive examples of the same form type. This yields the best SFT configuration: 100% nonzero rate and the highest average reward (Table 2).

SFT Configuration	Steps	Avg Loss	Nonzero Rate	Avg Reward
200 samples, lr=2×10^{-4}	13	0.075	98.4%	0.486
992 samples, lr=2×10^{-4} (overfit)	62	0.024	94.4%	0.426
992 samples, lr=5×10^{-5} (under-trained)	30	0.085	$\sim$55%	$\sim$0.30
992 samples, lr=10^{-4}, shuffled	**30**	**0.058**	**100.0%**	**0.492**

Table 2: SFT training ablation on 124 held-out validation tasks. Training on the full dataset with a high learning rate causes memorization and degrades generalization. A reduced learning rate with early stopping and data shuffling achieves the best performance. Note: these ablation results were evaluated with an earlier action parser; after parser improvements (collapsed matching, field aliases), the best SFT configuration achieves 0.614 avg reward (Table 1).

6.4 GRPO vs. SFT on Held-Out Tasks

Both the SFT model and the SFT+GRPO model achieve 100% nonzero reward rate—every single held-out task earns at least some reward. The difference lies in *how much* reward they earn. GRPO achieves higher average reward on both evaluation splits: 0.670 vs. 0.614 on validation (+9.1%) and 0.669 vs. 0.635 on test (+5.4%).

To understand *where* GRPO helps, we compare the two models task by task. On validation, GRPO outperforms SFT on 25 tasks, SFT outperforms GRPO on 20, and 79 are tied (both models earn the same reward). On test, it is nearly balanced: 18 wins for each model with 88 ties. However, GRPO's overall average is higher because when GRPO wins, it wins by a large margin (average gain of 0.368 per winning task), while SFT's wins are smaller (average gain of 0.116).

Statistical significance. To confirm that GRPO's improvement is not due to chance, we apply the Wilcoxon signed-rank test—a standard statistical test that checks whether paired observations (here, GRPO reward vs. SFT reward on each task) are systematically different. On the validation split, the difference is statistically significant ($p = 0.007$, well below the conventional threshold of $p < 0.05$). On the test split, the *two-sided* test—which asks "are GRPO and SFT different in either direction?"—is not quite significant ($p = 0.083$). However, the *one-sided* test—which asks the more specific question "is GRPO better than SFT?"—is marginally significant ($p = 0.041$). The

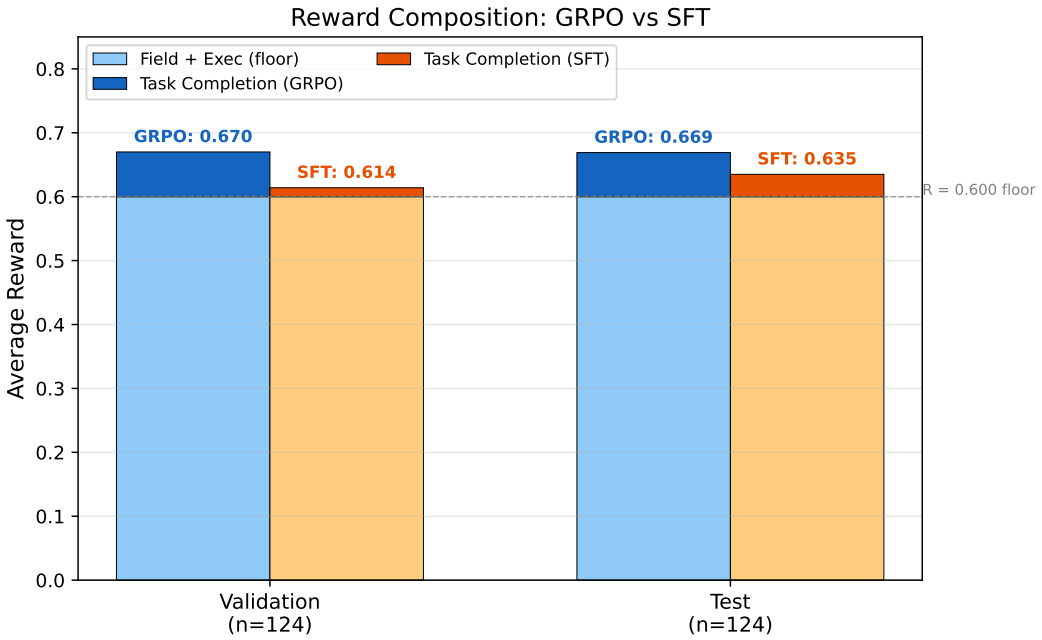


Figure 3: Reward composition breakdown. The base reward of 0.600 (the "floor") comes from perfect field accuracy ($R_{\text{field}} = 1.0$, weight 0.4) and full execution completeness ($R_{\text{exec}} = 1.0$, weight 0.2). The additional reward above the floor reflects successful task completion ($R_{\text{task}} = 1$, weight 0.4). GRPO achieves a larger task completion contribution than SFT on both evaluation splits.

weaker test-split result is consistent with the nearly balanced win/loss count and the smaller average difference.

GRPO's advantage is most visible at the top end: GRPO achieves 20 perfect scores (reward = 1.0) on both validation and test, compared to only 12 and 14 for SFT (Figure 4a). This means GRPO successfully submits more forms than SFT.

The 0.600 reward floor. A revealing pattern in the reward distribution (Figure 3) is a concentration of scores at exactly $R = 0.600$. To understand why, consider a form where the model fills every field perfectly and all actions execute, but the form is *not* successfully submitted (perhaps because the submit button was not clicked or a required field was missed). Plugging into Equation 5: $R = 0.4 \times \underbrace{0}_{\text{no submission}} + 0.4 \times \underbrace{1.0}_{\text{perfect fields}} + 0.2 \times \underbrace{1.0}_{\text{all actions ran}} = 0.600$. This floor tells us something important: *both models are already very good at filling fields correctly*. The gap between SFT and GRPO is not about filling fields better—it is about successfully submitting the form. GRPO converts more of these near-perfect 0.600-scoring attempts into successful 1.0-scoring submissions.

GRPO was trained on the full 992-prompt training set, initialized from the tuned SFT checkpoint (lr=10^{-4}, shuffled, 30 steps). Training was early-stopped after 255 prompts (50 gradient updates without improvement in running average reward), with a best running average reward of 0.783 at gradient update 47 (see Section 6.5 for details).

6.5 Training Dynamics

We monitor several metrics during GRPO training to understand how the model learns:

How far is the model drifting from SFT? We track the KL divergence between the GRPO-trained model and the original SFT model. KL divergence is a measure of how different two probability distributions are—in our case, how much the model's output behavior has changed from its SFT starting point. We want this to be small enough that the model does not "forget" the action format it learned during SFT, but large enough that it can improve. The KL divergence ranges from 0.037 on

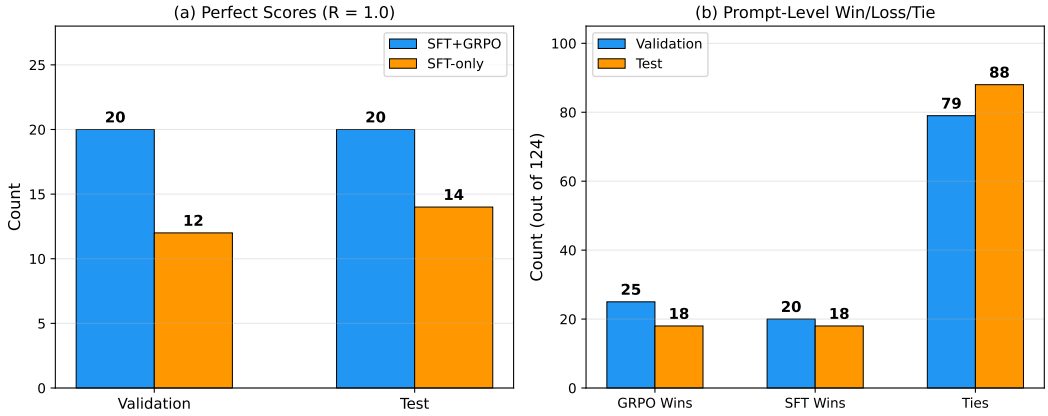


Figure 4: (a) Perfect score counts (reward=1.0): GRPO achieves more perfect scores than SFT on both splits, indicating better task completion. (b) Prompt-level win/loss/tie: most prompts are tied, but GRPO's wins are larger in magnitude (avg win margin 0.368 vs. SFT's 0.116).

simple forms (the model produces nearly the same output as SFT) to 0.681 on complex forms (the model has learned noticeably different strategies). We set the KL penalty coefficient to $\beta = 0.1$ to keep the model close to its SFT initialization. An earlier attempt with $\beta = 0.25$ was too restrictive: it penalized the model so heavily for deviating from SFT that the model could not improve.

How does reward progress during training? GRPO processes 255 of the 992 training prompts before early stopping kicks in. The running average reward (computed over a sliding window of 20 gradient updates) peaks at 0.783 at gradient update 47, then begins to decline. When no improvement is observed for 50 consecutive gradient updates, training stops automatically. Of the 255 prompts processed, only 97 (38.0%) produce gradient updates; the remaining 158 are skipped because all four candidates in the group receive nearly identical rewards (variance < 0.01), providing no useful learning signal.

Which forms benefit most from GRPO? Forms with higher within-group reward variance—meaning the four candidates score differently from each other—provide the strongest learning signal. We observe this mainly on "harder" forms with many dropdown fields or complex layouts, where different action orderings lead to meaningfully different outcomes.

Figure 5 visualizes the training dynamics. Panel (a) shows per-prompt average reward: gray dots are skipped prompts (low variance), blue dots produce gradient updates, and the star marks the best checkpoint. Panel (b) shows the KL divergence at each gradient update, illustrating the wide variation across forms.

6.6 Comparison with Commercial APIs

Commercial models like Gemini-2.5-Flash and GPT-4o achieve 98–100% success rates on FormFactory when paired with an agentic browser framework (Agent and CodeAgent modes), a dramatic improvement over the below-5% accuracy these same models achieve without such a framework. However, this comes with important caveats: these models are orders of magnitude larger than Qwen3-8B and require expensive API calls for every inference.

Our fine-tuned Qwen3-8B achieves 100% nonzero reward rate—every task receives at least partial reward. However, a direct comparison is difficult because the metrics are different: the commercial baselines report *binary success rate* (did the form submit successfully: yes or no?), while our metric gives partial credit for partially correct attempts. A task where the model fills 9 of 10 fields correctly but fails to submit would count as 0% in binary success but roughly 0.56 in our composite reward. The practical takeaway is that our 8-billion-parameter model, compressed to 4-bit precision, runs on a single consumer GPU with no API costs, making it suitable for high-volume or latency-sensitive deployments where calling a commercial API for every form would be prohibitively expensive.

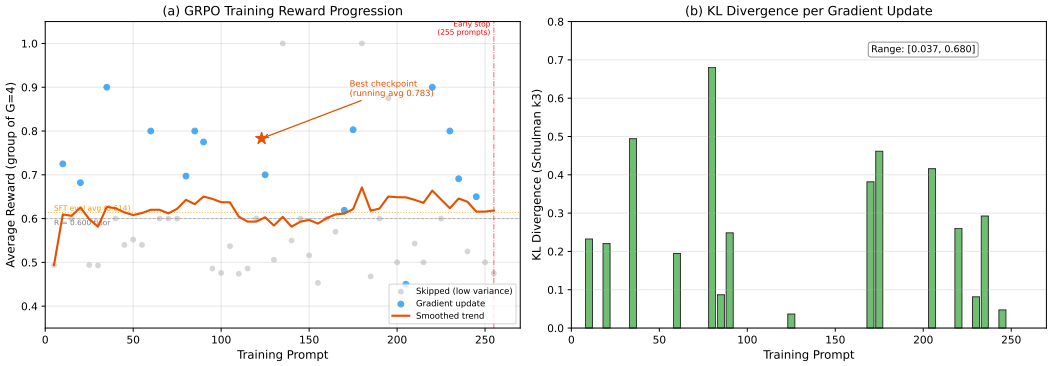


Figure 5: GRPO training dynamics. (a) Per-prompt average reward during training: gray points are skipped prompts (low within-group variance), blue points produce gradient updates. The star marks the best checkpoint (running average 0.783). (b) KL divergence varies substantially across prompts, with higher values on complex forms.

6.7 Per-Domain Analysis

Table 3 and Figure 6 break down performance by domain category across both evaluation splits. GRPO's improvement over SFT is concentrated in specific domains:

	Validation			Test		
Category	#	GRPO	SFT	#	GRPO	SFT
tech-software	10	**0.790**	0.590	10	**0.798**	0.634
finance-banking	15	**0.733**	0.627	15	**0.813**	0.707
arts-creative	15	**0.773**	0.690	15	**0.764**	0.751
healthcare-medical	14	**0.743**	0.686	14	**0.686**	0.657
academic-research	25	**0.666**	0.616	25	**0.666**	0.650
legal-compliance	15	**0.639**	0.634	15	**0.612**	0.602
construction-mfg	10	0.600	0.600	10	0.600	0.600
professional-biz	20	0.495	**0.501**	20	0.496	**0.503**

Table 3: Per-category average reward on validation and test splits. GRPO improves most on tech-software (+34% val, +26% test) and finance-banking (+17% val, +15% test). Construction-manufacturing forms are tied at the 0.600 floor (perfect field filling but no detected submission). Professional-business is the only category where SFT slightly outperforms GRPO.

Three patterns emerge from the per-domain analysis:

- **GRPO's gains come from submitting forms, not from filling fields better.** When GRPO beats SFT on a particular task, the improvement is almost always exactly $+0.400$—which is precisely the weight assigned to task completion in our reward function. In other words, GRPO learned how to successfully click "Submit" on forms where SFT fills all fields correctly but somehow fails to complete the submission. This is why GRPO's wins are large (average margin 0.368) while SFT's wins are small (average margin 0.116).
- **Some forms are impossible for both models.** Construction-manufacturing forms (order-request, project-bid) score exactly $R = 0.600$ for both models on every example. Both models fill all fields correctly, but the forms contain required file-upload fields (where users must attach a document). Since our agent cannot upload files, the browser's built-in validation blocks submission entirely. These forms always have zero reward variance during GRPO training and are always skipped.
- **GRPO slightly hurts on a few form types.** On professional-business forms (workshop-registration, startup-funding), SFT achieves marginally better scores than GRPO. On these

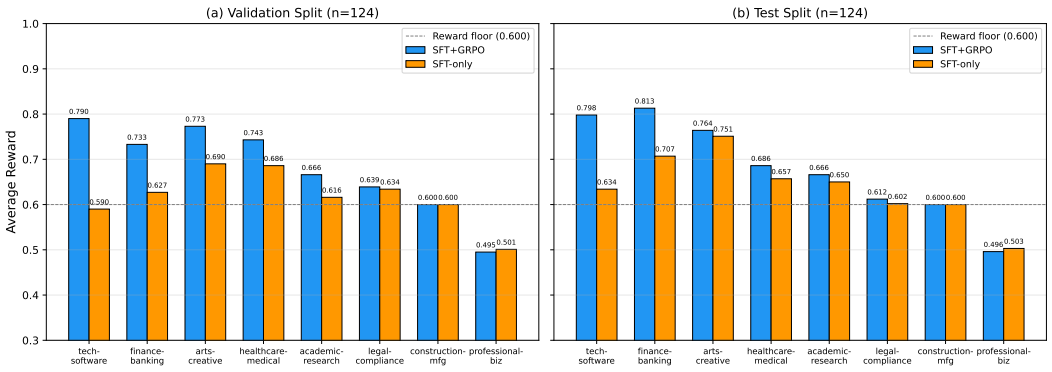


Figure 6: Per-domain average reward comparison between SFT+GRPO and SFT-only on validation (a) and test (b) splits. The dashed line marks the $R = 0.600$ reward floor. GRPO's largest gains are in tech-software and finance-banking; construction-manufacturing forms are stuck at the floor for both models.

complex forms, GRPO's exploration sometimes discovers worse action orderings than the original SFT demonstrations.

6.8 Evaluation Methodology

An important methodological lesson: **how you decode the model's output during evaluation matters a lot.** When we first evaluated our GRPO model using random sampling (temperature=0.7, which introduces randomness into the model's word choices), we saw only 95–97% nonzero rates and highly variable rewards (0.338–0.430 across different runs). Switching to greedy decoding (always picking the most likely next word, with no randomness) immediately improved results to 100% nonzero rate with stable rewards. This happened because random sampling occasionally causes the model to "hallucinate" invalid field names or skip actions, while greedy decoding consistently picks the model's best guess. The lesson: always use the same decoding strategy when comparing models, and prefer greedy decoding for deterministic, reproducible evaluations.

7 Discussion

7.1 Assessment of Hypotheses

We assess three key research questions:

Q1: Does GRPO achieve higher average reward than SFT on held-out tasks? Supported. GRPO achieves higher average reward than SFT on both held-out splits: +9.1% on validation (0.670 vs. 0.614) and +5.4% on test (0.669 vs. 0.635). Both achieve 100% nonzero reward rate, but GRPO produces more perfect scores (reward=1.0): 20 vs. 12 on validation and 20 vs. 14 on test. A paired Wilcoxon signed-rank test confirms statistical significance on the validation split ($p = 0.007$); the test-split improvement is marginally significant one-sided ($p = 0.041$) but not two-sided ($p = 0.083$). This suggests that online reinforcement fine-tuning with real browser execution can improve generalization beyond imitation learning, with stronger evidence on the validation split than on the test split.

Q2: Does GRPO learn better form submission strategies than SFT? Supported. GRPO's improvement comes entirely from the task completion component—learning to successfully submit forms where SFT falls short—rather than from better field filling. Both models fill fields with comparable accuracy, but GRPO converts more $R = 0.600$ near-misses (where all fields are filled correctly but submission fails) into $R = 1.0$ successful submissions. GRPO achieves 20 perfect scores (reward=1.0) on both validation and test, compared to only 12 and 14 for SFT. When GRPO outperforms SFT on a task, the margin is consistently $+0.400$—exactly the task completion weight—indicating that GRPO discovers submission strategies absent from the SFT demonstrations. This

finding is consistent with the reinforcement learning paradigm: by practicing on real forms, the model learns action sequences that lead to successful submission through trial and error, complementing the format knowledge acquired during SFT.

Q3: Does real browser execution provide more informative reward signal than simple text comparison? Supported. The online reward function, which uses actual browser execution outcomes, provides richer feedback than simply comparing the model's predicted text against a reference answer. For example, the field accuracy component measures the values *actually submitted* to the form (after the browser executes the actions), which may differ from the model's text output if an action fails to execute. The execution completeness component directly measures whether the browser can interpret and carry out the model's planned actions—information that is impossible to obtain from text comparison alone.

7.2 Why SFT Is Necessary

The failure of GRPO without SFT reveals a fundamental challenge: reinforcement learning can only refine behaviors that the model *already knows how to do*, at least approximately. For math or code tasks, a pre-trained LLM already produces outputs in a roughly correct format (equations, code). But web agent action plans ("Step 1: Type 'Name' with 'John' ") are a domain-specific format that Qwen3-8B has never seen during pre-training. When GRPO tries to learn from scratch, the model generates random text, earns zero reward on everything, and has no signal to improve—a classic "cold start" problem.

This is a well-known challenge in RL: policy gradient methods need a reasonable starting point that produces *some* positive reward, otherwise there is nothing to reinforce. SFT provides this starting point by teaching the model the action format. GRPO can then improve the *content* of the actions (which fields to fill, what values to enter, when to click submit) within the format that SFT established.

7.3 Reward Function Design

The composite reward function (Section 4.4) provides informative feedback at multiple granularities. Each component addresses a different failure mode:

- **Task completion** (40% weight) answers the big question: did the form actually submit? This is the ultimate goal.
- **Field accuracy** (40% weight) gives credit for partial progress. A model that fills 8 of 10 fields correctly receives a field accuracy score of roughly 0.8, even if the form does not submit. This is critical for learning: without it, a model that fills 8 fields correctly and one that fills 0 fields would receive the same reward (both failing to submit), leaving no signal for improvement.
- **Execution completeness** (20% weight) discourages the model from "making up" actions that cannot be executed. If the model references a field called "Middle Name" but the form has no such field, this action fails and execution completeness drops.

The continuous field accuracy component is especially important for GRPO. Since GRPO compares candidates within a group, it needs reward *variation* between candidates. Binary success alone would produce many ties (all candidates either succeed or fail); the continuous field accuracy score creates the variation needed for meaningful advantage computation.

7.4 Limitations

Our study has several limitations:

1. **High GRPO skip rate:** GRPO can only learn when the candidates in a group receive different rewards. During training, 62.0% of prompts were skipped because all four candidates produced nearly identical outputs (and therefore nearly identical rewards), leaving no signal for improvement. We tried generating candidates at different temperatures (varying the randomness), but the SFT checkpoint is so well-trained that it produces the same text regardless of temperature. Generating more candidates per prompt (increasing G beyond 4) might help.

2. **Single benchmark:** All experiments use FormFactory, a synthetic form-filling benchmark. Generalization to real-world forms with diverse layouts, CAPTCHAs, and dynamic content is not tested.
3. **Action parser limitations:** Our field-name matching is imperfect. It handles spelling variations and common abbreviations, but fails on purely semantic mismatches (e.g., the model outputs "Street Address" but the form's HTML uses `current_street`). The parser also cannot handle file uploads, which blocks submission on forms that require document attachments. These limitations affect all models equally and set an upper bound on achievable performance.
4. **Single model architecture:** We evaluate only Qwen3-8B. The SFT/GRPO comparison may yield different results on other model families or scales.
5. **Single-turn evaluation only:** Although we implemented a multi-turn mode where the model observes intermediate DOM states after each action, we only evaluated the single-turn approach (generating a complete action plan in one shot) due to computational cost (estimated 72 hours per run) and browser stability constraints. Evaluating the multi-turn pipeline remains future work.
6. **Single training run:** All results are from single training runs with a fixed random seed. While the paired Wilcoxon signed-rank test confirms statistical significance on the validation split ($p = 0.007$), the test-split improvement is only marginally significant one-sided ($p = 0.041$) and not significant two-sided ($p = 0.083$). Multiple runs with different seeds would strengthen these conclusions.
7. **No reward function ablation:** The composite reward weights (0.4, 0.4, 0.2) were selected based on domain intuition and not ablated. We did not compare against a binary-only reward to isolate the contribution of dense feedback.

Broader impacts. Web automation agents could potentially be misused for fraudulent form submissions, automated creation of fake accounts, or spam. Our released training pipeline is general-purpose: while the fine-tuned model is specialized to FormFactory's synthetic forms and cannot generalize to arbitrary websites without additional training, the SFT+GRPO methodology could be retrained on real-world forms. We release the pipeline for research purposes and encourage responsible use on controlled benchmarks. The model's reliance on a local form server and a fixed action format limits out-of-the-box misuse, but practitioners adapting this work to production settings should implement appropriate safeguards such as rate limiting and human-in-the-loop review.

8 Conclusion

We studied whether reinforcement learning can improve a language model's ability to fill out web forms, beyond what supervised learning alone achieves. Using Qwen3-8B fine-tuned with QLoRA on the FormFactory benchmark, we found four main results:

1. **SFT is necessary before GRPO:** Without first teaching the model the correct output format via SFT, GRPO fails completely—the model does not know how to produce action plans, so it earns zero reward on every attempt and has nothing to learn from.
2. **SFT training dynamics matter:** Training too aggressively causes the model to memorize exact training examples rather than learning general patterns, which hurts performance on new forms. A reduced learning rate, early stopping, and data shuffling prevent this.
3. **GRPO improves over SFT:** Starting from the SFT checkpoint, GRPO achieves 9.1% higher average reward on validation and 5.4% on test. The improvement comes from better form submission, not better field filling—GRPO learns to complete the final submission step on forms where SFT falls short.
4. **Dense reward matters:** A reward function that gives partial credit for partially correct attempts (rather than just binary success/failure) provides the variation needed for GRPO to distinguish better candidates from worse ones within each group.

The fine-tuned Qwen3-8B achieves at least partial reward on all 248 held-out tasks while running on a single consumer GPU, showing that parameter-efficient fine-tuning makes web agents practical without expensive cloud infrastructure.

Future Work. The most promising direction is reducing GRPO's 62% skip rate—the fact that most training prompts produce identical candidates and cannot contribute to learning. Generating more candidates per prompt or injecting random perturbations into the model's actions could create the diversity needed for learning. A second direction is evaluating our implemented *multi-turn mode*: instead of planning all actions in advance, the model observes the form after each action and decides what to do next, enabling genuine error detection and correction. This mode is implemented in our released codebase but was not evaluated due to the estimated 72-hour training time and browser stability constraints. Other directions include evaluation on additional benchmarks and improving the action parser to handle more complex form layouts.

Acknowledgments and Disclosure of Funding

This work was completed as part of a graduate research project at the University of Toronto Scarborough. Compute resources were provided through AWS g5.xlarge instances on Anyscale Ray. We thank the FormFactory team for their open-source benchmark.

References

Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., and Su, Y. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*, 2025.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*, 2021.

Li, B., Wang, Y., Fei, H., Li, J., Ji, W., Lee, M.-L., and Hsu, W. FormFactory: An Interactive Benchmarking Suite for Multimodal Form-Filling Agents. *arXiv preprint arXiv:2506.01520*, 2025.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y., Wu, Y., and Guo, D. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., et al. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.

Putta, P., Mills, E., Garg, N., Motwani, S., Finn, C., Garg, D., and Rafailov, R. Agent Q: Advanced Reasoning and Learning for Autonomous AI Agents. *arXiv preprint arXiv:2408.07199*, 2024.

Bai, H., Yin, Y., Ravichander, A., Chen, Y., Bisk, Y., Sap, M., and Neubig, G. DigiRL: Training In-The-Wild Device-Control Agents with Autonomous Reinforcement Learning. *arXiv preprint arXiv:2406.11896*, 2024.

Qi, J., Xu, J., Pan, M., Zhang, Y., Gong, Y., Song, D., and Liu, Q. WebRL: Training LLM Web Agents via Self-Evolving Online Curriculum Reinforcement Learning. *arXiv preprint arXiv:2411.02337*, 2024.

Williams, R. J. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 8(3):229–256, 1992.

Yao, S., Chen, H., Yang, J., and Narasimhan, K. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., and Neubig, G. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations (ICLR)*, 2024.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract states GRPO achieves 9.1% higher average reward on validation and 5.4% on test, which matches Table 1 results exactly.

   Guidelines:

   - The answer NA means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 7.4 discusses seven specific limitations including high skip rate, single benchmark, parser limitations, single model, single-turn evaluation only (multi-turn implemented but not evaluated), single training run, and no reward ablation.

   Guidelines:

   - The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: This is an empirical paper with no theoretical results or proofs.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 5 provides complete experimental setup including all hyperparameters, dataset splits, compute infrastructure, and evaluation methodology.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The complete training pipeline (SFT trainer, GRPO trainer, evaluation scripts) is released at `https://github.com/billy-enrizky/openbrowser-ai`. Trained LoRA adapters are available on HuggingFace: `https://huggingface.co/billyenrizky/Qwen3-8B-FormFactory-SFT-LoRA` and `https://huggingface.co/billyenrizky/Qwen3-8B-FormFactory-GRPO-LoRA`.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so "No" is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [Yes]

Justification: Section 5 details the dataset (FormFactory, 1240 tasks, 80/10/10 split), all model hyperparameters, QLoRA configuration, and evaluation protocol.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We report paired Wilcoxon signed-rank tests with exact p-values for both validation and test splits, including both one-sided and two-sided tests.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Section 5.4 specifies all compute: single NVIDIA A10G (24GB), AWS g5.xlarge on Anyscale Ray, with training time estimates.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work trains web agents on synthetic form-filling benchmarks with no real user data, PII, or ethical concerns.

   Guidelines:

   - The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Section 7.4 discusses potential misuse of web automation agents (fraudulent form submissions, fake accounts) and mitigation strategies (rate limiting, human-in-the-loop review). The released pipeline is intended for research on controlled benchmarks.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

    Answer: [NA]

    Justification: The trained models operate only on FormFactory benchmark forms and cannot generalize to arbitrary web tasks without additional fine-tuning.

    Guidelines:

    - The answer NA means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: FormFactory is properly cited. All model architectures (Qwen3, QLoRA) and algorithms (GRPO, REINFORCE) are cited with original papers.

    Guidelines:

    - The answer NA means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.

- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The training pipeline is documented with configuration details and usage instructions.

    Guidelines:

    - The answer NA means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [NA]

    Justification: No crowdsourcing or human subjects research was conducted.

    Guidelines:

    - The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [NA]

    Justification: No human subjects research was conducted.

    Guidelines:

    - The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.

- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: The core method fine-tunes an LLM (Qwen3-8B) for web form-filling, which is the primary subject of the paper.

    Guidelines:

    - The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy (`https://neurips.cc/Conferences/2025/LLM`) for what should or should not be described.